Reducing Inference Latency and Improving Filtering Accuracy in Lightweight, Privacy-Preserving Email Triage: A Case Study on Skyler AI

Sanjay Kumar

***Abstract*—**Inbox-management products increasingly rely on small, locally-run Natural Language Processing (NLP) models to triage, summarize, and classify email, as an alternative to routing message content through third-party cloud Large Language Models (LLMs). We take as a direct baseline three recent, verifiable works that each evaluate one part of this problem — triage-action prediction [1], thread summarization [2], and business-workflow classification [3] — and show that each validates its own narrow task offline, on a fixed dataset, without measuring what a deployed product actually needs: request-time serving cost, the accuracy of a single *combined*, multi-signal decision (not one model's label), or data-locality guarantees. We close this gap in Skyler AI, a production Gmail inbox-organization application that filters, prioritizes, and drafts replies to email using local DistilBERT and MiniLM models plus a cloud LLM used only for user-initiated reply drafting. We identify and fix a model-reinitialization bug that wasted 252-310 ms on every classification request; we identify and fix a miscalibrated semantic-similarity threshold that was silently degrading a combined sentiment-and-content filtering rule to 60.5% F1; and we build and pass a network-level test proving zero data egress during classification. We report classification quality on the real, official SST-2 benchmark (89.91% accuracy, 90.27% F1) and end-to-end filtering-decision accuracy on a labeled test set (94.4% accuracy, 90.9% F1), both measured against the real, deployed system, alongside an explainability mechanism absent from all three baselines.

***Index Terms*—**email triage, inbox management, on-device NLP, inference latency, privacy-preserving AI, explainable AI, similarity threshold calibration

I. INTRODUCTION

Email remains the primary communication channel for most knowledge work, and inbox overload is a well-documented productivity problem. Skyler AI addresses this directly as a multi-inbox organizer: it lets a user define a filtering rule per inbox (sentiment, keyword, and semantic-similarity conditions), runs that rule against newly pulled Gmail messages using local models, and offers one-click AI-drafted replies for the messages that remain. The classification and filtering path runs entirely on local models (a sentiment classifier and a sentence embedder); a cloud LLM is invoked only afterward, for the smaller, user-initiated task of drafting a reply to an email the user has already chosen to act on. This split is a privacy- and cost-motivated design choice, consistent with the design philosophy argued for in recent inbox-management literature [1], [3], [6].

This paper makes four contributions. First, we compare Skyler AI's design directly against three verifiable, recent papers that each cover one part of this same problem space — triage, summarization, and workflow classification — and show precisely what none of them measures. Second, we identify and fix two concrete, previously unmeasured bugs in Skyler AI's own deployed code: a request-time model-reinitialization cost, and a miscalibrated semantic-similarity threshold that was silently reducing filtering recall by more than half. Third, we add explainability and a proven data-locality guarantee, both absent from the cited baselines. Fourth, we report all results honestly, including two optimizations that did not survive measurement and were not shipped.

II. RELATED WORK AND RESEARCH GAP

A. Direct literature baseline

Table I summarizes three directly relevant papers, each covering one of the three product behaviors Skyler AI combines: deciding what to do with an email, summarizing it, and classifying it for a business workflow. Mukherjee and Jiang [1] predict whether a user will act on (e.g., reply to) an incoming email, using content features plus user/email similarity features, evaluated on the public Avocado email collection; their best ensemble reaches 0.788 AUROC against a 0.757 best-baseline. Zhang et al. [2] introduce EmailSum, a human-annotated dataset for abstractive email-thread summarization, and report that common automatic metrics (ROUGE, BERTScore) correlate only weakly with human judgment on this task — notably, they decline to present a single trusted accuracy number for this reason. Prexawanprasut [3] classifies real small-business email into four operational workflow categories using transformer models, reporting that XLM-R exceeds 92% accuracy on 12,500 emails drawn from four business domains.

TABLE I: Literature Baseline Summary

Paper	Model	Task	Reported Metric	What It Measures
[1] Mukherjee and Jiang	TF-IDF/GBDT + CNN-MLP ensemble	Predict user triage action (e.g., reply)	0.788 AUROC (best), vs. 0.757 best baseline	Offline ranking quality, one public dataset
[2] Zhang et al. (EmailSum)	T5/BART abstractive summarization	Email thread summarization	No single trusted automatic metric; authors show ROUGE/BERTScore correlate weakly with humans	Summary quality via human + automatic eval, fixed dataset
[3] Prexawanprasut	BERT, DistilBERT, XLM-R	Business email classification (4 workflow categories)	XLM-R greater than 92% accuracy, 12,500 emails	Offline classification accuracy, fixed labeled dataset

B. Broader literature

Rojas-Galeano [4] compares Flan-T5 against GPT-4 for zero-shot spam classification, reporting a model-size/accuracy tradeoff (90% vs. 95% F1) without addressing request-time serving — relevant here only as supporting evidence for preferring small local models over large cloud LLMs. Kamberi [5] pairs a MiniLM embedding model with keyword extraction for email routing, again evaluated offline. Distil Labs [8] offers a distilled local classifier explicitly for privacy, again evaluated only on offline accuracy. Kabakibo et al. [6] and Liu et al. [7] study cold-start latency, but at the scale of GPU-backed LLM-serving engines and distributed edge pipelines, not small in-process models embedded in ordinary application code. A broader literature on embedding-similarity calibration [10] independently confirms that fixed cosine-similarity thresholds do not transfer across models or domains and must be calibrated against labeled data — directly relevant to Section IV-C.

C. The gap

Across all three baseline papers, a model is validated for its own single task — triage-action ranking, summarization, or category classification — offline, on a fixed dataset, with no deployed, request-driven product behind it. None measures: (1) the cost of initializing and serving that model on a live backend; (2) whether a *combined*, multi-signal product decision (sentiment and keyword and semantic match together, not one model's label) is correct; or (3) where user data goes during inference, despite privacy being a stated motivation for small local models over cloud LLMs in this space. This is the gap this paper addresses, in a single deployed system that must do all three things — triage, summarize, and classify — at once, in production, per request.

III. SYSTEM OVERVIEW

Skyler AI is a Gmail inbox-organization product: it gives each user multiple independent inboxes, each with its own filtering rule, and a credits-based usage model (each refresh of up to 200 messages costs one credit). The backend (Express/TypeScript) runs two local models through an ONNX runtime: a DistilBERT sentiment classifier and a MiniLM sentence embedder, both executed in-process with no GPU. A cloud LLM (Groq Llama-3.1-8b-instant) is used only for drafting a reply to a single, user-selected email — never for the classification path evaluated in this paper. The relevant request path fetches up to 200 Gmail messages per user action and, if an AI filter rule is configured, classifies them using a rule combining sentiment, literal keyword matching, and semantic similarity; matched emails can then be summarized and replied to in one click, without leaving Gmail.

IV. METHODOLOGY

A. Warm model singleton

The deployed classifier re-initialized both ONNX models from scratch on every classification request, discarding them at the end of the request, despite an existing but unused process-wide singleton loader already present in the codebase. We replace per-request instantiation with the warm singleton, and warm it once at process startup.

B. Length-bucketed batching

Naive batching of embedding and sentiment-analysis calls (one call for an entire set of texts) was measured to erase the benefit of batching for embeddings and to regress sentiment-analysis latency by approximately 8% on realistic, variable-length email text, because padding every text to the batch's longest member costs more than batching saves. We instead sort texts by length and batch within small, length-similar buckets (size 16), measured as a consistent ~1.1-1.2x improvement over both sequential calls and naive full-batching.

C. Calibrated similarity threshold

The deployed default similarity threshold for the semantic-matching path was 0.6. We constructed a labeled test set (90 synthetic emails; ground truth assigned by construction across a sentiment-by-topic grid, not manual judgment) and measured the deployed system's filtering decision against it. At the shipped default, precision was 100% but recall was 43.3% (F1 60.5%), because genuinely on-topic paraphrases measured real cosine similarities of 0.25-0.54 against single-word rule keywords — almost all below the 0.6 cutoff. A threshold sweep identified 0.34-0.36

as a genuine local optimum (verified with a fine-grained sweep, not a single lucky value); we ship 0.35.

D. Explainability

We add a method returning, for every email, the reason behind its filtering decision (detected sentiment label, matched keyword if any, and semantic-match boolean), exposed on the live endpoint as an additive field alongside the existing response contract.

E. Data-locality verification

We instrument a test that patches every Node.js network entry point (HTTP, HTTPS, raw TCP, TLS, and the global fetch API) before loading the classification models, then executes a full model-load-and-classification cycle, asserting zero network calls.

V. EVALUATION

A. Model-reinitialization cost

Isolated measurement on the project's cached model weights shows 252-310 ms of pure model-loading overhead eliminated per request by the warm singleton, reduced to approximately 0 ms after process warm-up.

B. Realistic-scale, end-to-end latency

A reproducible benchmark harness drives the actual production classifier against a faithful reconstruction of the pre-fix classifier, at the product's own stated scale (200 emails/request). Across three independent runs, mean speedup varied from 1.01x to 1.37x on a shared, non-dedicated machine, but latency variance was consistently reduced by 1.2x-8.2x, and worst-case latency spikes were bounded from 11-14 seconds down to approximately 8 seconds. We report the range transparently rather than a single favorable run.

C. Classification quality

Running the actual deployed sentiment model against the real, official SST-2 validation set (872 labeled examples) yields 89.91% accuracy and 90.27% F1. This is a different task and a different label space than any of [1]-[3], so no baseline accuracy is directly comparable; the conceptually closest reference point is [3]'s business-email classification (XLM-R, greater than 92% accuracy, 4 categories), which classifies email into workflow categories rather than deciding keep/discard — stated here as a parallel data point, not a direct comparison.

D. End-to-end filtering-decision accuracy

Against the labeled 90-email test set described in Section IV-C, the calibrated system achieves 94.4% accuracy, 100% precision, 83.3% recall, and 90.9% F1 for the combined sentiment-and-keyword-and-semantic filtering decision — a metric none of the three baseline papers reports, since each evaluates one model's offline output (a ranking score, a summary, or a category label) rather than a deployed, combined, multi-signal decision. Per-error diagnosis attributes four of the five remaining errors to the sentiment sub-model's domain mismatch (trained on movie reviews; email administrative phrasing such as "still needs processing" independently scores as negative sentiment at high confidence), consistent with and explaining the 89.91% ceiling in Section V-C, rather than to any remaining calibration or batching defect.

E. Privacy verification

The network-interception test described in Section IV-E passes with zero recorded calls across model loading, sentiment analysis, and semantic embedding, providing a proof rather than an assertion of the data-locality claim motivating this and related work [8].

TABLE II: Skyler AI vs. the Literature Baseline

Axis	Baseline ([1],[2],[3])	Skyler AI (measured)
Classification quality	0.788 AUROC (triage); no single metric (summarization); greater than 92% acc. (workflow, 4-class)	89.91% acc./90.27% F1 (sentiment, different task)
Request-time serving cost	Not measured by any of the three	252-310 ms/request eliminated
Combined filtering-decision accuracy	Not measured by any of the three	94.4% acc., 90.9% F1
Explainability (multi-signal)	None of the three expose a per-decision reason	Full reason per email, live
Data locality / privacy	Not measured or proven by any of the three	Proven: 0 network calls
Latency predictability	Not measured by any of the three	1.2x-8.2x variance reduction

VI. DISCUSSION AND LIMITATIONS

The filtering-accuracy test set is self-constructed, since no public labeled "should this rule keep this email" dataset exists; validation against real, held-out email traffic is future work. The realistic-scale latency benchmark was run on one shared, non-dedicated machine; a multi-machine study would narrow the reported range. The sentiment-model domain-mismatch finding (Section V-D) is diagnosed but deliberately not patched with an unvalidated heuristic, to avoid overfitting a fix to our own five-example error set. Live production `classifyMs` telemetry exists in the shipped code but has not yet been collected against real Gmail

traffic, since this evaluation environment has no access to a live account or production database.

VII. CONCLUSION AND FUTURE WORK

We identified a three-part gap in recent inbox-management literature — serving-time cost, combined-decision accuracy, and data-locality guarantees — none of which is measured by three directly comparable papers covering triage, summarization, and workflow classification. We closed this gap in a real, deployed system: fixing a 252-310 ms per-request latency bug, fixing a similarity-threshold calibration bug that had cut filtering recall by more than half (43.3% to 83.3% recall after calibration), adding explainability for a combined decision, and proving — not asserting — a zero-data-egress privacy guarantee. Future work includes a held-out, real-email-traffic validation of the filtering-accuracy benchmark, a multi-machine latency study, and investigation of sentence-weighted sentiment scoring to address the diagnosed domain-mismatch ceiling, validated on data independent of this paper's own test construction.

REFERENCES

1. S. Mukherjee and K. Jiang, "A Content-Based Approach to Email Triage Action Prediction: Exploration and Evaluation," arXiv:1905.01991, 2019.
2. S. Zhang, A. Celikyilmaz, J. Gao, and M. Bansal, "EmailSum: Abstractive Email Thread Summarization," in Proc. 59th Annual Meeting of the Association for Computational Linguistics and the 11th Int. Joint Conf. Natural Language Processing (ACL-IJCNLP 2021), arXiv:2107.14691, 2021.
3. T. Prexawanprasut, "Transformer-Based Email Classification for Workflow Automation in Small and Medium-Sized Enterprises," Thailand Statistician, 2025, open access via TCI-Thaijo.
4. S. Rojas-Galeano, "Zero-Shot Spam Email Classification Using Pre-trained Large Language Models," in Applied Computer Sciences in Engineering (WEA 2024), Communications in Computer and Information Science, vol. 2222, Springer, 2024. arXiv:2405.15936.
5. H. Kamberi, "Intelligent Email Routing with LLMs," M.Sc. thesis, South East European Univ., Tetovo, North Macedonia, 2026.
6. H. S. Kabakibo, A. Trivedi, and L. Wang, "Breaking the Ice: Analyzing Cold Start Latency in vLLM," arXiv:2606.07362, 2026.
7. X. Liu, N. Xue, R. Bao, Y. Sun, Z. Chen, M. Tao, X. Xu, and S. Cui, "CSGO: Generalized Optimization for Cold Start in Wireless Collaborative Edge LLM Systems," arXiv:2508.11287, 2025.
8. Distil Labs, "distil-email-classifier" (model card), accessed via featherless.ai, 2025.
9. Xenova / Hugging Face, "distilbert-base-uncased-finetuned-sst-2-english" and "all-MiniLM-L6-v2" (ONNX model ports).
10. S. Anand, "Embeddings similarity threshold," s-anand.net blog, 2024 (and related calibration literature discussed in Section II-B).